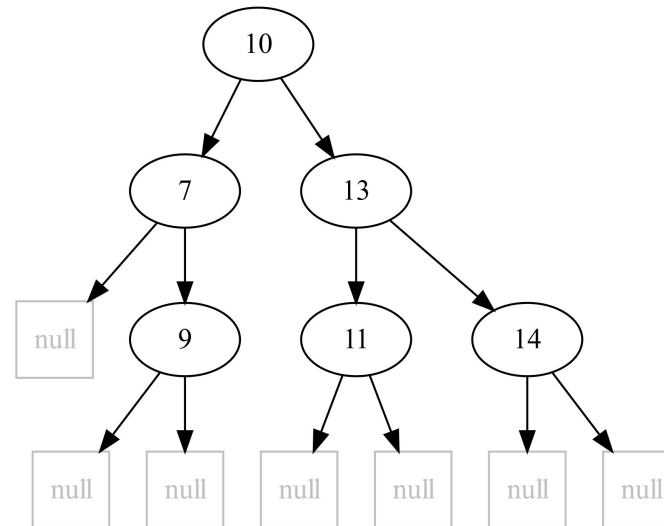


3.3 – Les arbres binaires de recherche (ABR)

Définition

- Un arbre binaire de recherche (ABR) est un arbre binaire qui vérifie les propriétés suivantes :
 - Chaque nœud possède une clé
 - Pour chaque nœud, tous les nœuds du sous-arbre gauche ont une clé inférieure (ou égale), et tous les nœuds du sous-arbre droit ont une clé (strictement) supérieure
- Exemple :



Remarque

- Un parcours infixe d'un arbre binaire de recherche retourne les valeurs triées

Recherche dans un ABR

- Algorithme de recherche dans un arbre binaire de recherche :
 si arbre = vide alors
 retourner NOT_FOUND
 sinon
 si valeur-recherchée = valeur-nœud alors
 retourner FOUND
 sinon
 si valeur-recherchée < valeur-nœud alors
 retourner recherche(valeur-recherchée, sous-arbre-gauche)
 sinon
 retourner recherche(valeur-recherchée, sous-arbre-droit)
 finsi
 finsi
 finsi

Complexité de la recherche dans un ABR

- Complexité de la recherche en $O(h)$ dans le pire des cas, avec h la hauteur de l'arbre
 - Si l'arbre est équilibré, la hauteur de l'arbre devient minimale (ce qui correspond à $O(\log(n))$) => la complexité dans le pire des cas est $O(\log(n))$
 - Mais une succession d'insertions et de suppressions peut déséquilibrer un arbre, aboutissant à une hauteur $h = O(n)$, et ainsi à une complexité de la recherche en $O(n)$ dans le pire des cas

Insertion dans un ABR

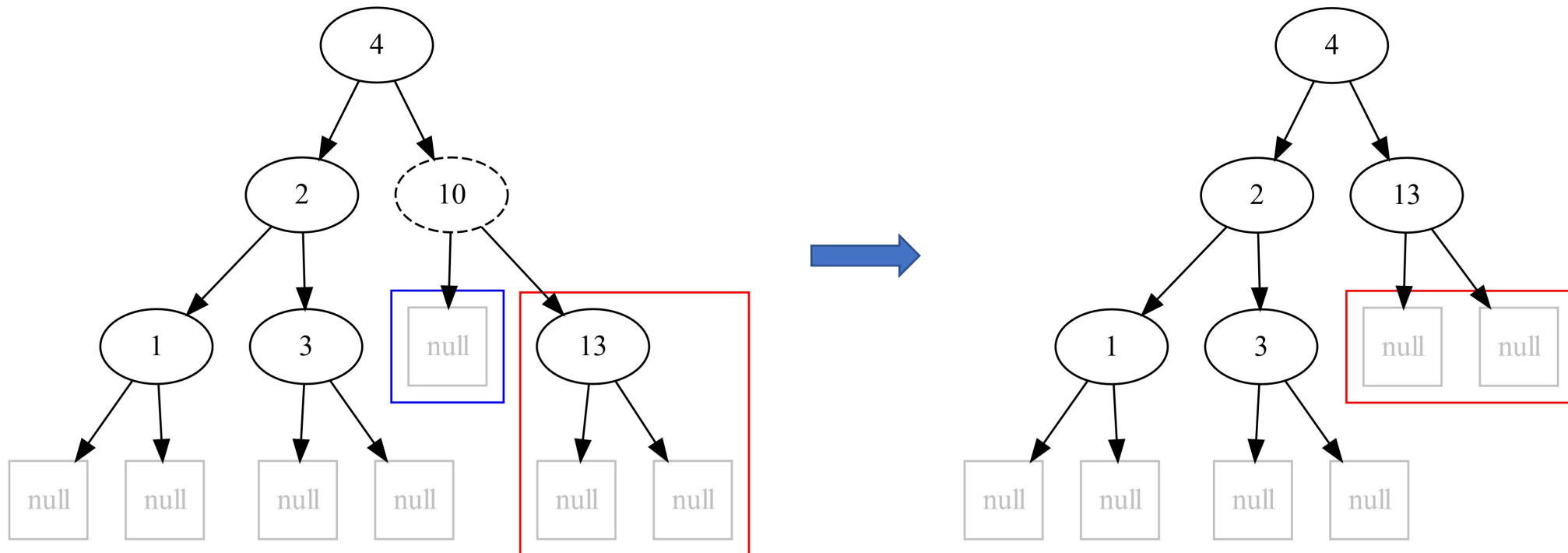
- Algorithme d'insertion d'une clé c dans un ABR :
 si arbre=vide alors
 arbre $\leftarrow$ créer nœud(c)
 sinon
 si $c < \text{racine}(a)$ alors
 si fils-gauche(a) est vide alors fils-gauche(a) $\leftarrow$ créer nœud(c)
 sinon insertion(fils-gauche(a), c)
 sinon
 si $c > \text{racine}(a)$ alors
 si fils-droit(a) est vide alors fils-droit(a) $\leftarrow$ créer nœud(c)
 sinon insertion(fils-droit(a), c)
 finsi
 finsi
 finsi

Suppression dans un ABR (1/3)

- Algorithme (principe) :
 - On recherche tout d'abord le nœud contenant la valeur
 - Si le nœud est une feuille => il suffit de la supprimer
 - Si le nœud est un nœud interne => il faut le supprimer et raccorder les deux potentiels fils
- Pour raccorder les deux fils suite à la suppression de n :
 - Soit on remplace n par le nœud du sous-arbre gauche ayant la plus grande clé
 - Soit on remplace n par le nœud du sous-arbre droit ayant la plus petite clé

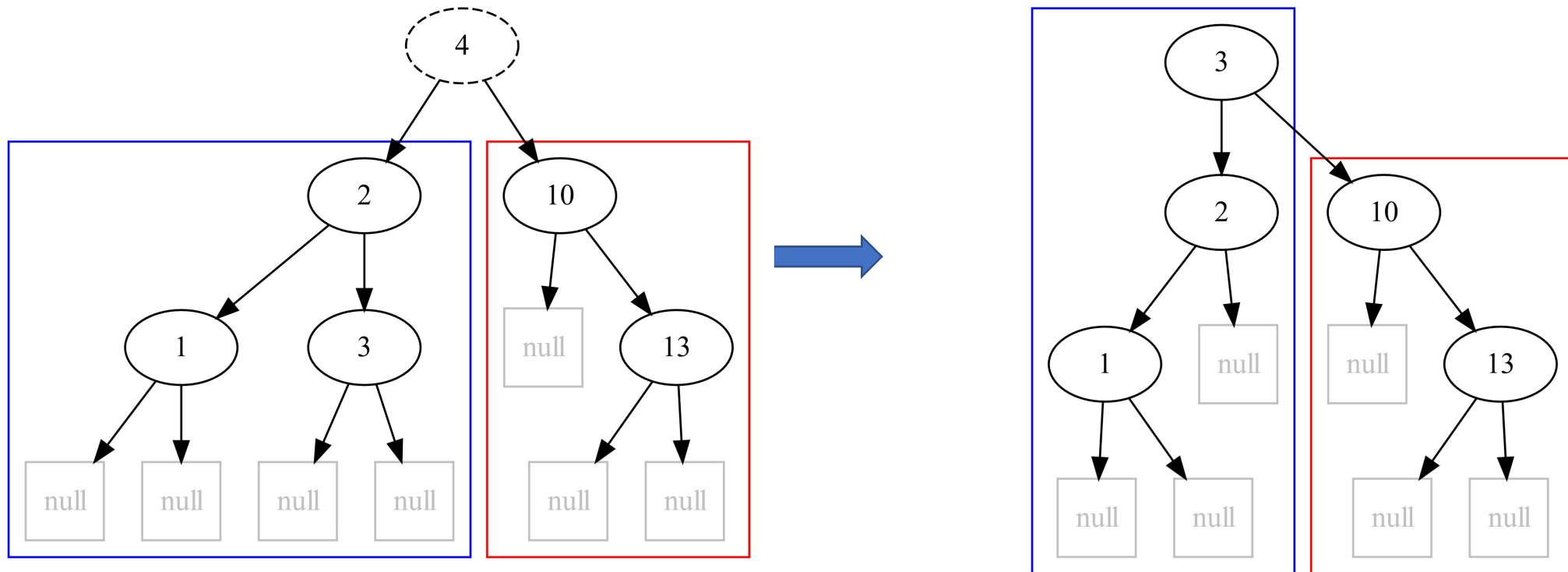
Suppression dans un ABR (2/3)

- Exemple de raccordement : on supprime le nœud $n=10$ et on le remplace par le nœud du sous-arbre droit ayant la plus petite clé



Suppression dans un ABR (3/3)

- Exemple de raccordement : on supprime le nœud n=4 et on le remplace par le nœud du sous-arbre gauche ayant la plus grande clé



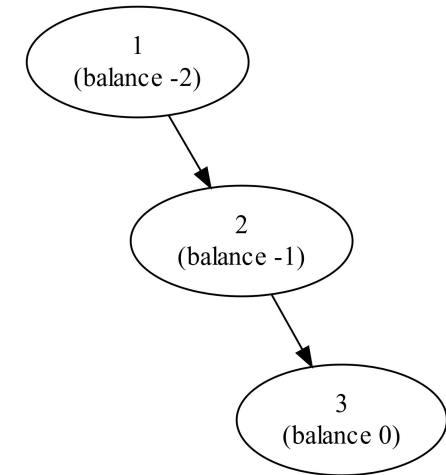
Minimum/maximum dans un ABR

- La valeur minimum dans un ABR est obtenue en cherchant la feuille la plus à gauche
- La valeur maximale dans un ABR est obtenue en cherchant la feuille la plus à droite

3.3.1 – Les arbres AVL

Définition

- Arbre AVL = inventés en 1962 par Adelson-Velsky et Landis
- Un arbre AVL est un ABR respectant une propriété supplémentaire :
 - $balance(n) = hauteur(fils-gauche(n)) - hauteur(fils-droit(n))$
 - Pour chaque nœud n , $|balance(n)| \leq 1$
- Pour garantir cette propriété, les opérations d'insertion et de suppression doivent parfois rééquilibrer l'arbre
- Complexité des opérations
 - Un AVL est toujours équilibré, ce qui permet de garantir que la recherche est en $O(\log(n))$ dans le pire des cas
 - L'insertion et la suppression dans un AVL sont aussi en $O(\log(n))$



Recherche dans un AVL

- La recherche dans un AVL est identique à la recherche dans un ABR

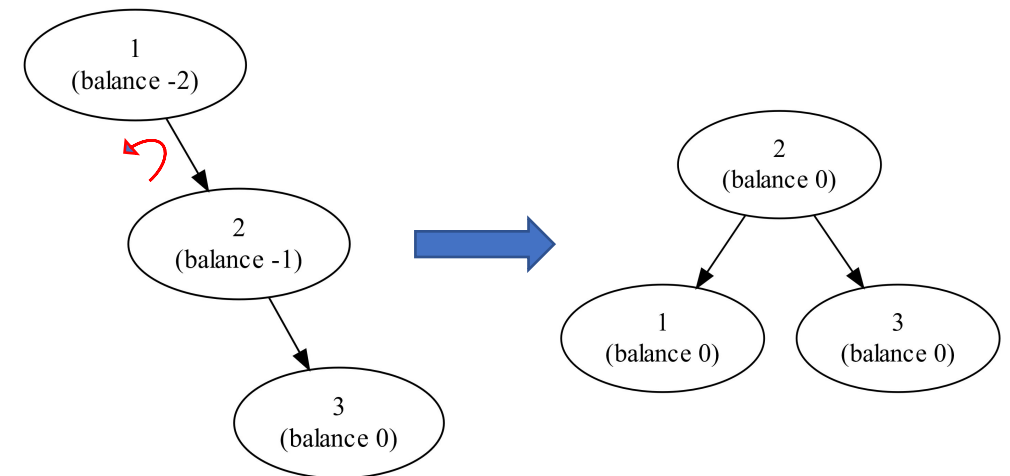
Insertion dans un AVL

- On recherche l'endroit où ajouter le nouveau nœud en suivant l'algorithme de recherche
 - Le nouveau nœud remplace toujours une feuille vide
- On remonte les ancêtres du nouveau nœud pour vérifier si l'arbre doit être rebalancé
 - Si $|\text{balance}(\text{père})| \leq 1$ alors on arrête le rééquilibrage
 - Si $|\text{balance}(\text{père})| = 2$ alors on effectue une « rotation » du sous-arbre pour rééquilibrer
- Remarque : au plus une rotation (simple ou double) est nécessaire pour rééquilibrer un arbre AVL

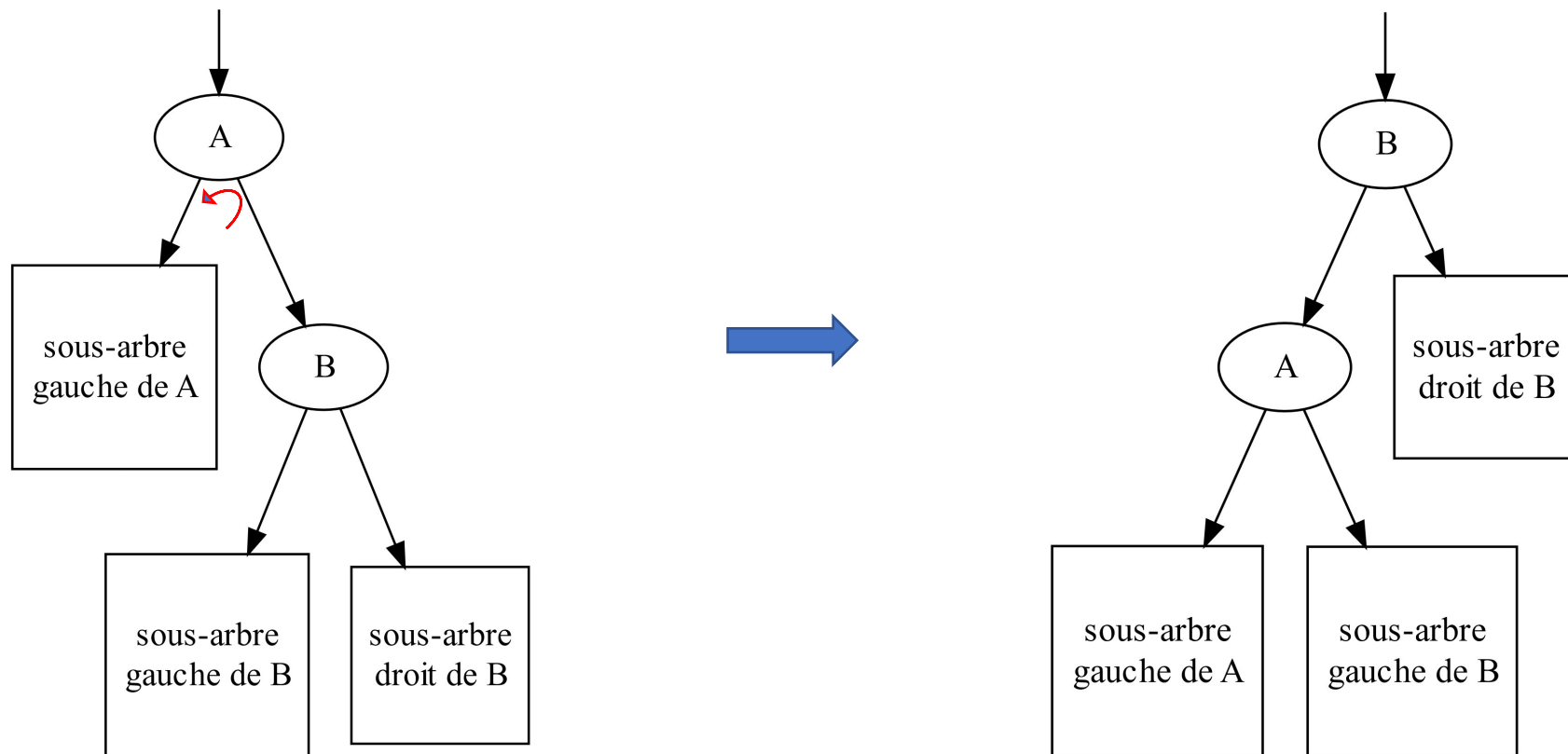
Quelle rotation effectuer ?

- Quatre types de rotation sont utilisées pour rééquilibrer un AVL :

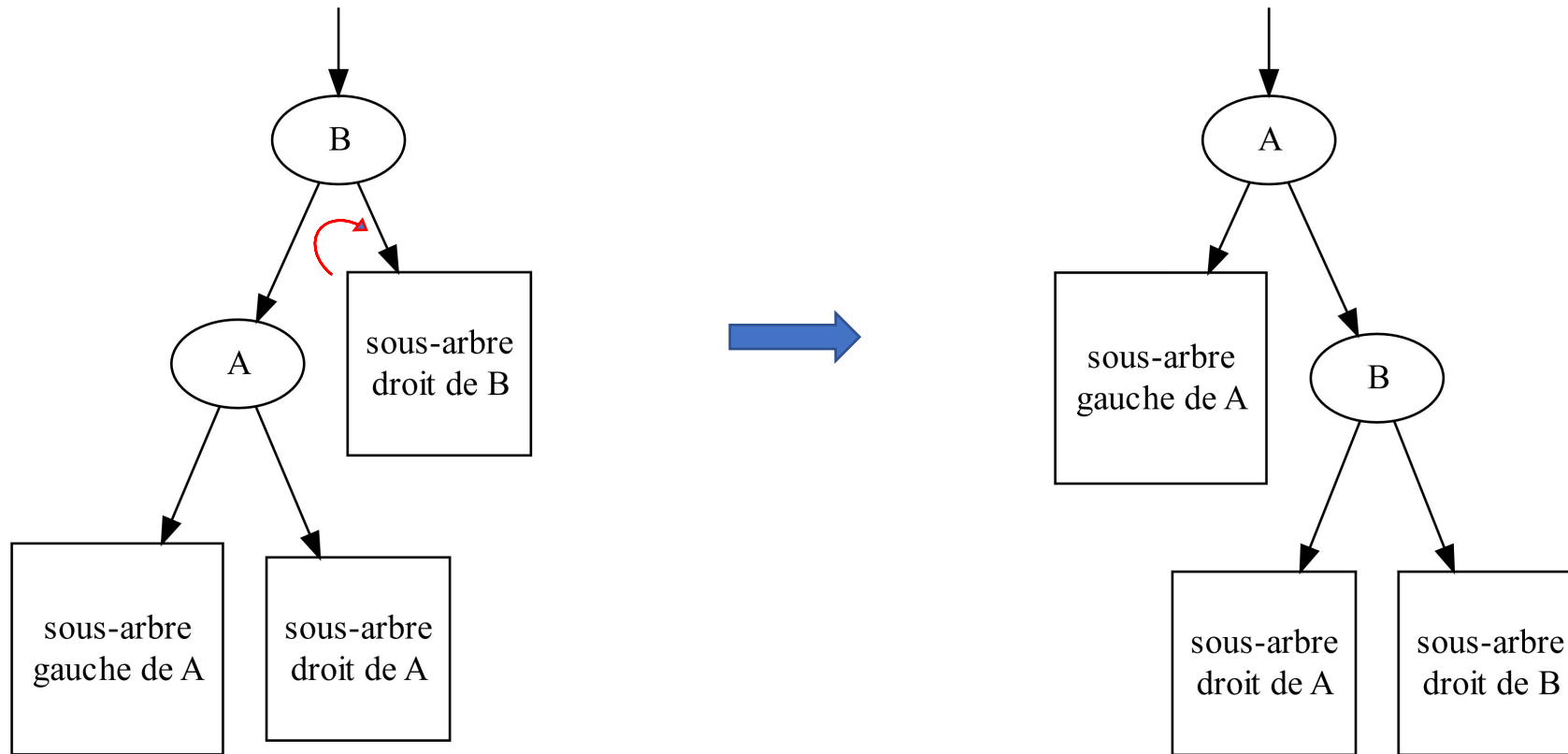
si n est le fils droit de son père alors
 si balance(père)=-2 alors
 si balance(n)>0 alors
 rotation droite-gauche
 sinon
 rotation gauche
 finsi
 finsi
sinon si balance(père)=2 alors
 si balance(n)<0 alors
 rotation gauche-droite
 sinon
 rotation droite
 finsi
finsi



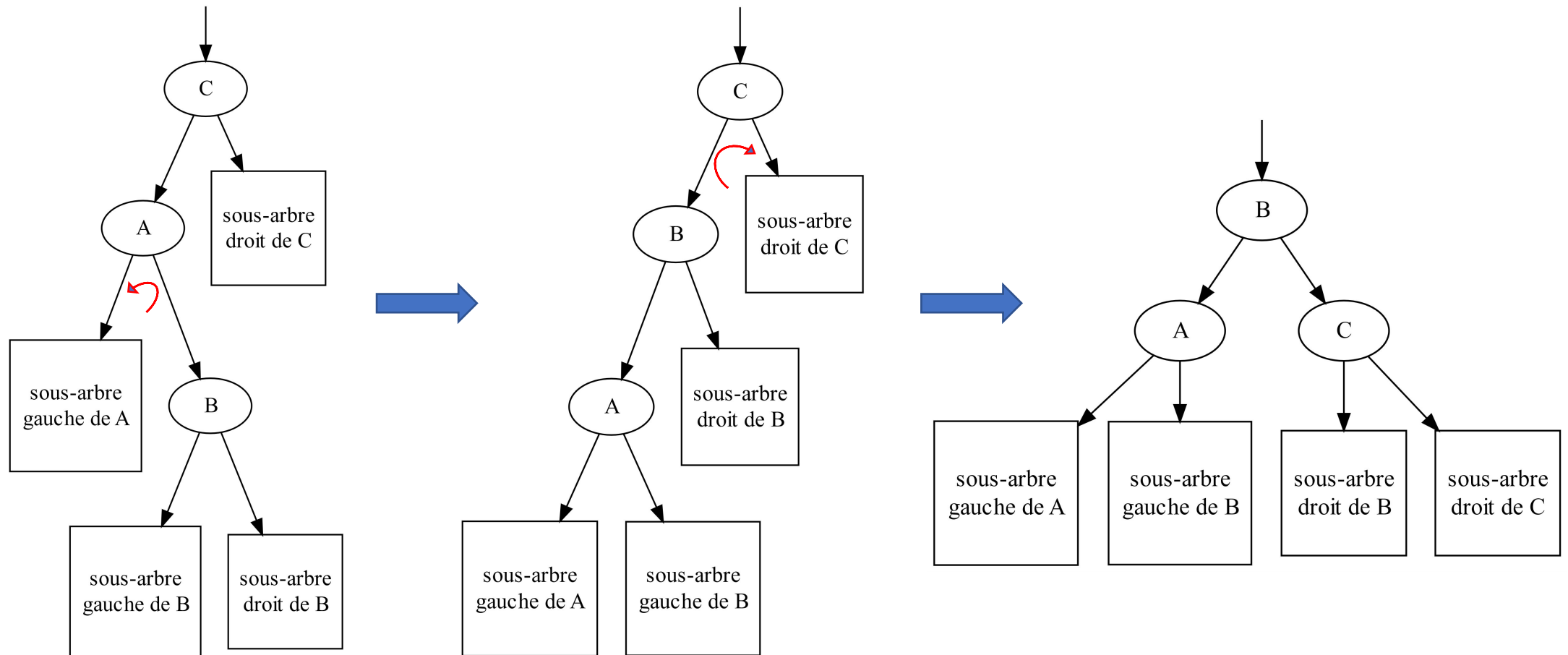
Rotation gauche dans un AVL



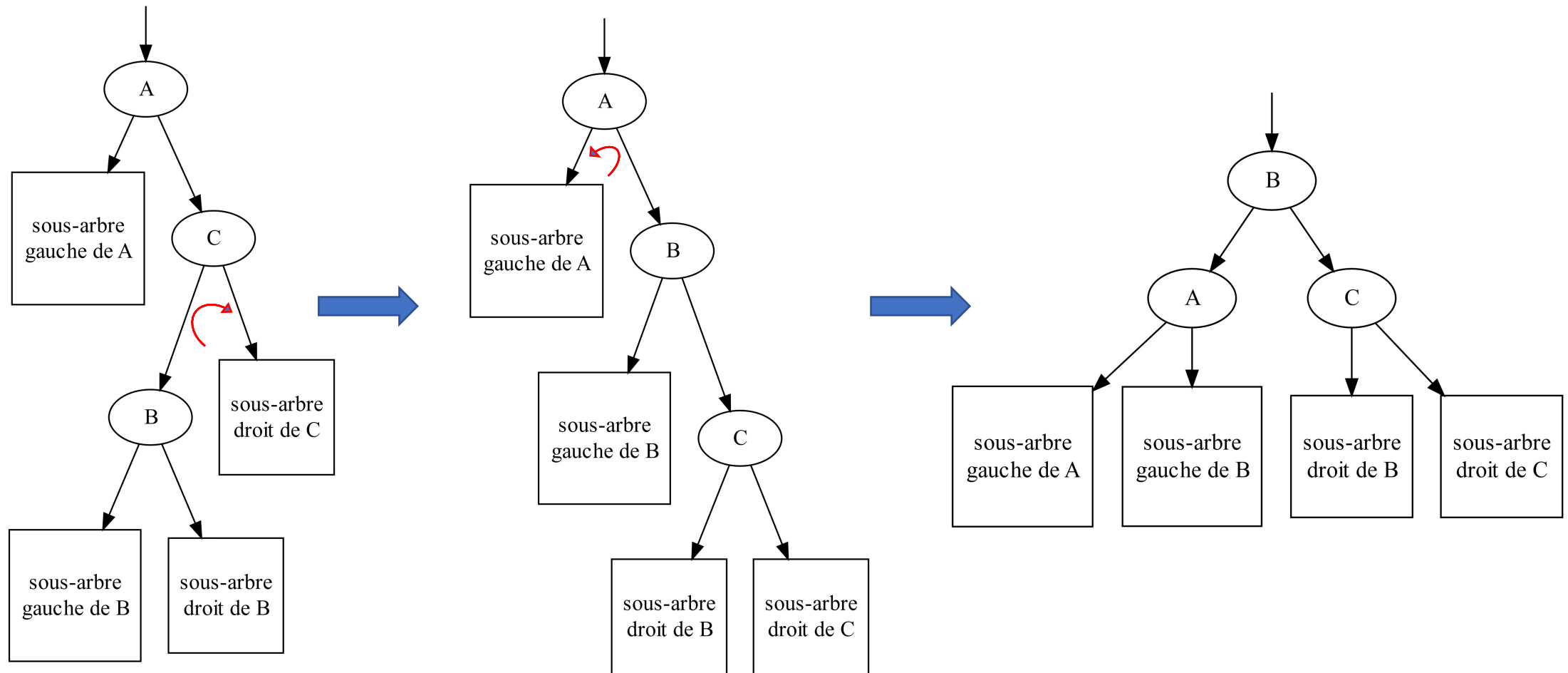
Rotation droite dans un AVL



Rotation gauche-droite dans un AVL



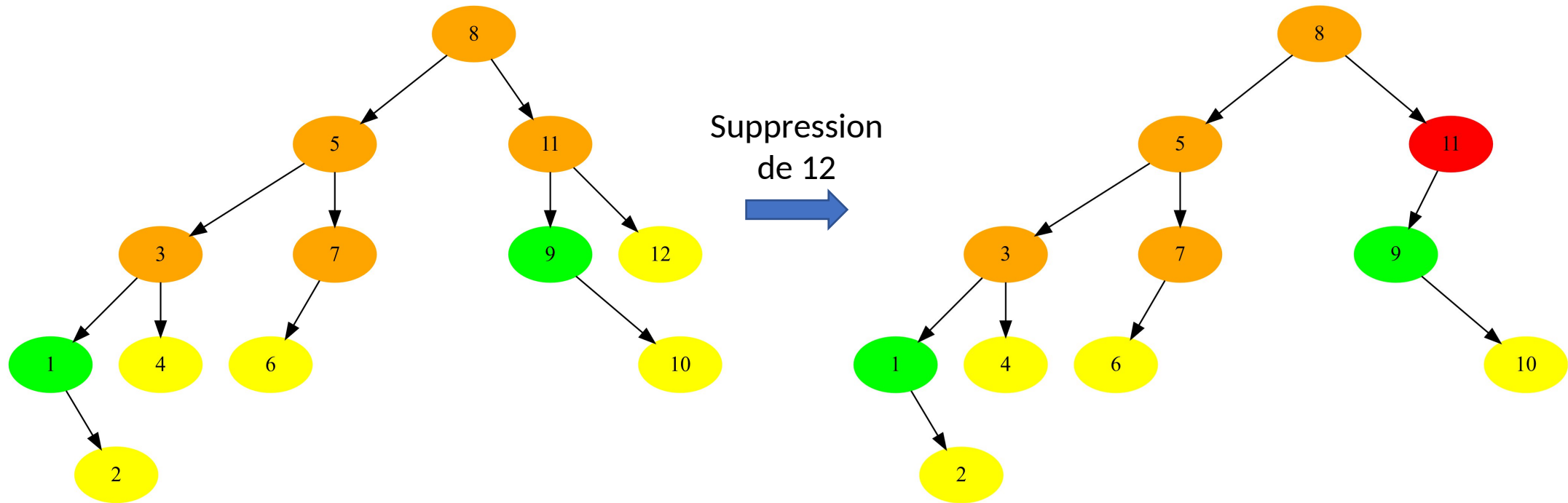
Rotation droite-gauche dans un AVL



Suppression dans un AVL

- Pour supprimer un nœud dans un AVL :
 - On effectue la même suppression que dans un ABR
 - Puis on remonte tous les ancêtres du nœud effacé pour vérifier que chaque ancêtre est équilibré
- Remarque : après une suppression (et contrairement aux insertions), il peut être nécessaire d'effectuer plusieurs rotations pour rééquilibrer l'arbre

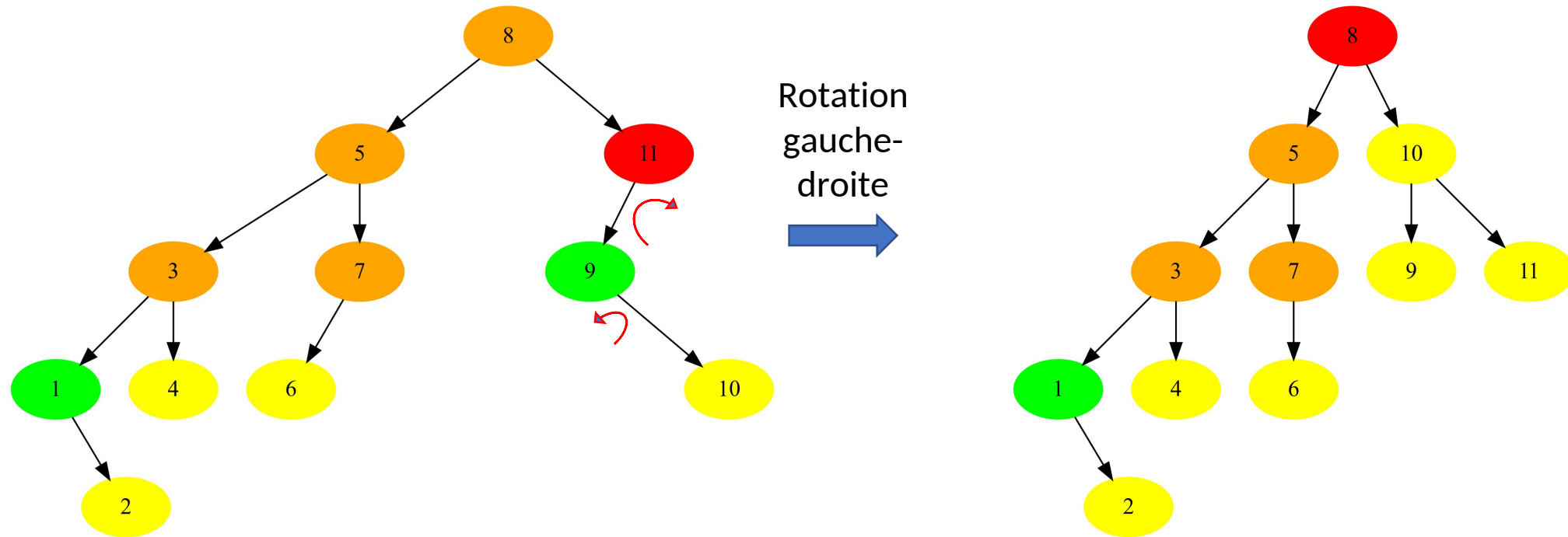
Exemple d'AVL dont la suppression nécessite plusieurs rotations : suppression de 12 (1/3)



Remarque : la couleur du nœud indique la balance (rouge pour 2, 1 pour orange, 0 pour jaune, -1 pour vert, -2 pour bleu)

Exemple redessiné à partir de <https://i.stack.imgur.com/lvGsS.png> (9/06/2023)

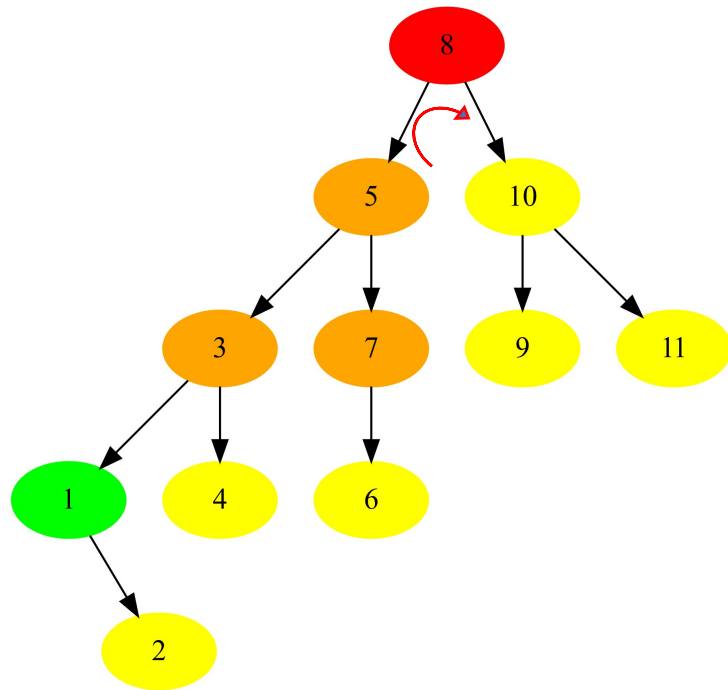
Exemple d'AVL dont la suppression nécessite plusieurs rotations : suppression de 12 (2/3)



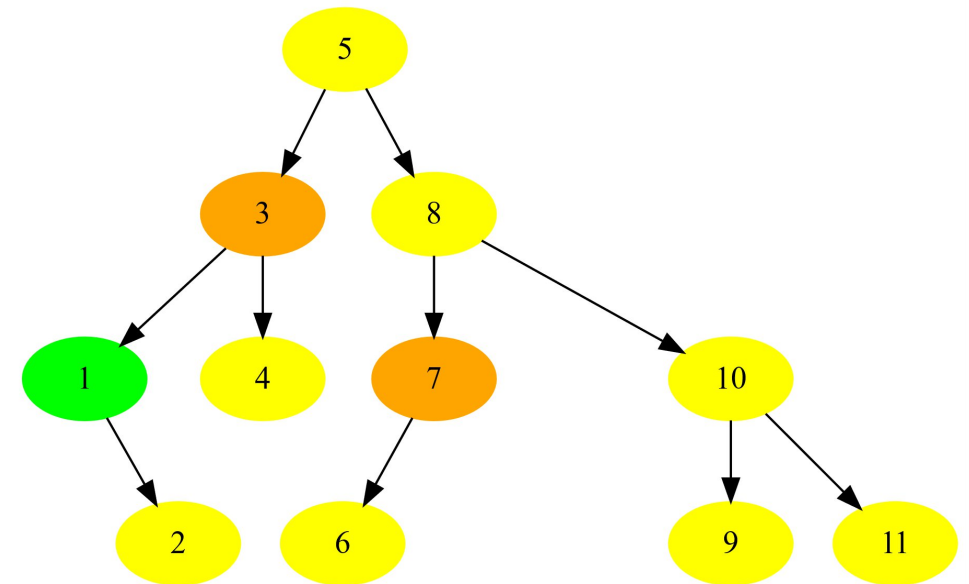
Remarque : la couleur du nœud indique la balance (rouge pour 2, 1 pour orange, 0 pour jaune, -1 pour vert, -2 pour bleu)

Exemple redessiné à partir de <https://i.stack.imgur.com/lvGsS.png> (9/06/2023)

Exemple d'AVL dont la suppression nécessite plusieurs rotations : suppression de 12 (3/3)



Rotation
droite
➡

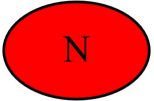


Remarque : la couleur du nœud indique la balance
(rouge pour 2, 1 pour orange, 0 pour jaune, -1
pour vert, -2 pour bleu)

Exemple redessiné à partir de
<https://i.stack.imgur.com/lvGsS.png>
(9/06/2023)

3.3.2 – Les arbres rouge-noir

Définition

- Les arbres rouge-noir ont été inventés en 1978 par L. J. Guibas et R. Sedgwick
- Un arbre rouge-noir est un arbre binaire de recherche équilibré
 - Chaque nœud a une couleur : rouge () ou noir ()
 - Un nœud rouge ne peut pas avoir un fils rouge
 - Chaque branche allant jusqu'aux feuilles traverse le même nombre de nœuds noirs (ce qui est appelé la « hauteur noire »)
 - Généralement, on considère que la racine est noire

Propriétés

- Dans un arbre rouge-noir, les opérations de recherche, d'insertion et de suppression sont en $O(\log(n))$

Pourquoi utiliser un arbre rouge-noir plutôt qu'un AVL ?

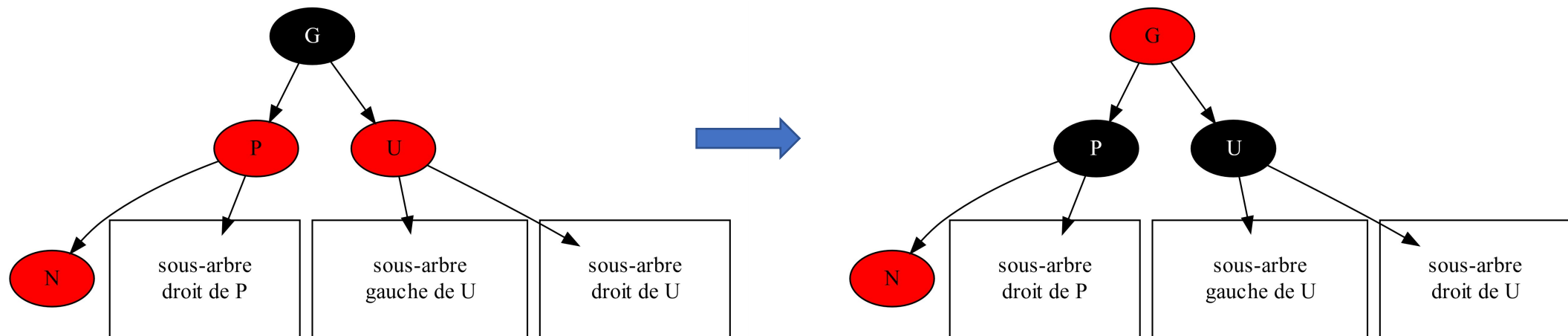
- Similitudes :
 - Les AVL et les arbres rouge-noir sont des ABR équilibrés
 - Les AVL et les arbres rouge-noir ont la même complexité dans le pire des cas pour la recherche, l'insertion et la suppression : $O(\log(n))$
- Différence :
 - Dans les AVL, la complexité amortie de l'insertion et de la suppression est en $O(\log(n))$
 - Dans les arbres rouge-noir, la complexité amortie de l'insertion et de la suppression est en $O(1)$; les rotations sont donc moins fréquentes en pratique
- Remarque : complexité amortie = considère les opérations globalement plutôt que le pire des cas (car le pire des cas n'arrive pas à chaque fois)
 - Ressemble à une complexité moyenne, mais sans notion d'aléatoire
 - On considère par exemple le nombre total d'opérations pour ajouter n éléments (au lieu de considérer les éléments un par un), et on le divise par n

Insertion (1/4)

- L'insertion dans un arbre rouge-noir :
 - On crée un nœud N rouge (pour ne pas impacter la règle de la longueur des chemins en nombre de nœuds noir) avec la clé
 - On l'insère à la position où la clé aurait été trouvée (donc sur une feuille)
 - On vérifie et corrige les éventuelles violations de la règle des nœuds rouges
- Il y a six cas pour vérifier/corriger une violation de la règle
 - Cas 1 : N est la racine => rien à faire
 - Cas 2 : P est noir (N n'est pas la racine) => rien à faire
 - Cas 3 : P est la racine (N n'est pas la racine + P est rouge) => P devient noir

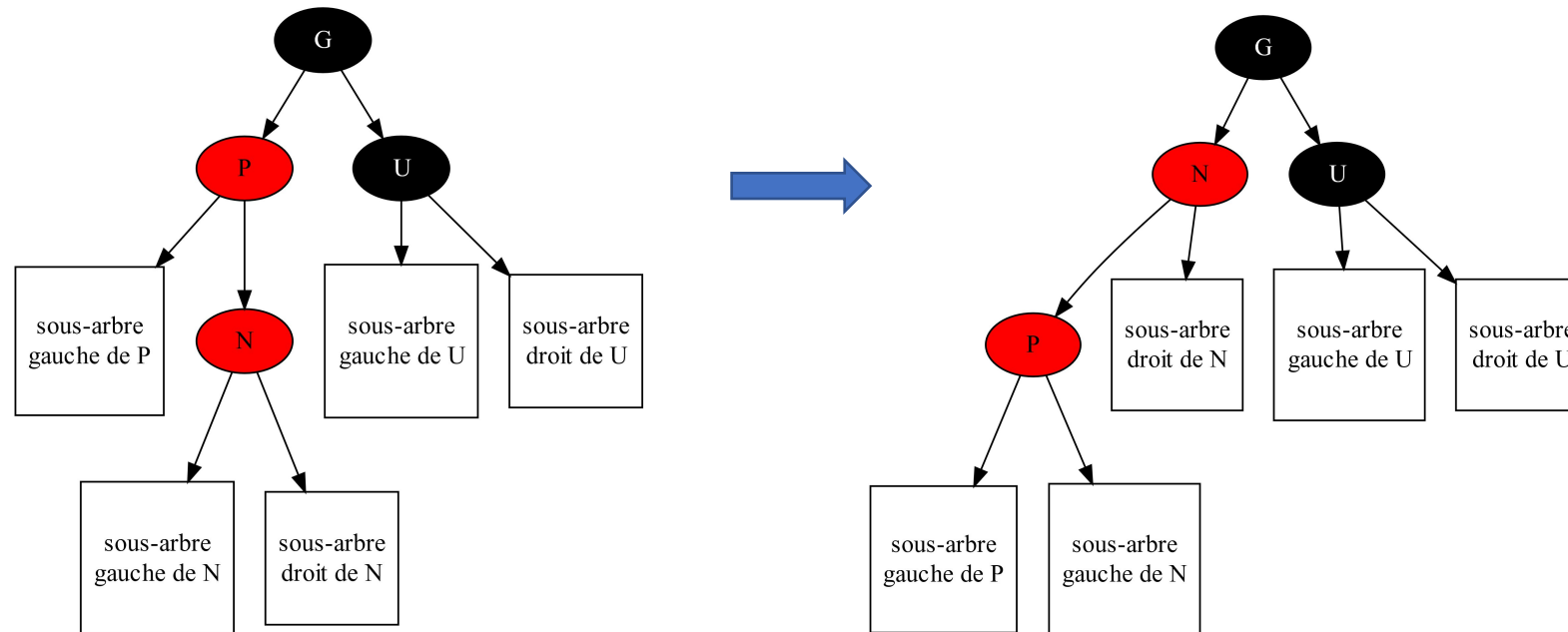
Insertion (2/4)

- Cas 4 : U est rouge (N n'est pas la racine + P est rouge + P n'est pas la racine) => G devient rouge, P et U deviennent noirs
 - Remarque : si U n'existe pas, on le considère noir
 - Après la modification : on repart de G



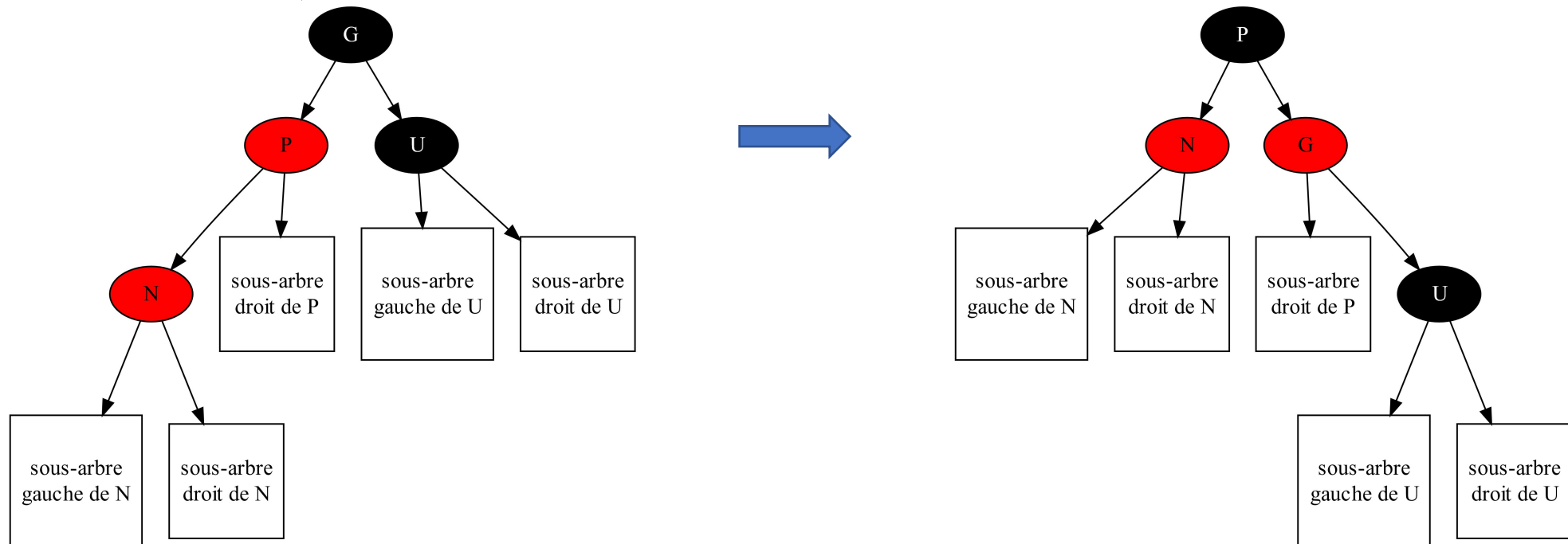
Insertion (3/4)

- Cas 5 : N est un nœud interne (= fils-gauche du fils-droit de G, ou fils-droit du fils-gauche de G) (N n'est pas la racine + P est rouge + P n'est pas la racine + U est noir) => on fait une rotation, puis on repart du nœud P



Insertion (4/4)

- Cas 6 : (N n'est pas la racine + P est rouge + P n'est pas la racine + U est noir + N est un nœud externe (= fils-gauche du fils-gauche de G, ou fils-droit du fils-droit de G)) => on fait une rotation pour inverser le rôle de G et P, et on inverse les couleurs de G et P



Exemple d'applications

- Exemple d'applications :
 - L'ordonnanceur Linux par défaut utilise un arbre rouge-noir (pour la classe « normale »)
 - Certains langages fonctionnels utilisent des arbres rouge-noir pour les tableaux associatifs
 - Dans Java 8, la classe HashMap utilise un arbre rouge-noir pour stocker les collisions (au lieu d'une liste dans les versions précédentes)